Comparison of Reinforcement Learning Techniques for Movie Recommendations

Nicole Falicov, Nanxi Liu, Angela Shen

1 Introduction

In the modern era of online entertainment, recommender systems play a large role in the media that people choose to watch. Improving recommendation systems allows people to more easily discover new media they can connect with and explore topics they are interested in. This is beneficial for both filmmakers and viewers. Robust and personalized recommendations can help individuals discover lesser-known content that they may not have encountered otherwise, as well as allow under-represented stories to reach a wider audience. The goal of our project is to develop a reinforcement learning recommendation system capable of generating successful personalized recommendations. For each user, our aim is to maximize their satisfaction with regards to the media that they are recommended. For our solution, we aim to train an agent through two reinforcement learning methods to provide recommendations and compare the outcome of the two methods. We chose to use reinforcement learning techniques since they are well-suited to maximizing long-term user engagement, compared to approaches that provide a single classification or prediction.

2 Background Related Work

2.1 Approaches to Recommendation Systems

There are many approaches to movie recommendation systems that do not take advantage of reinforcement learning. These techniques can be split into two general categories – collaborative filtering, which uses the preferences from users with similar taste to recommend new media, and content-based filtering, which matches user profiles with movie descriptions. Collaborative filtering is subject to data sparsity, scalability, and gray sheep (where the preference of a user significantly deviates from that of the majority). Content-based filtering, on the other hand, has a hard time dealing with new users who have little profile information. Its other shortcomings include limited content analysis and serendipity (referring to the generation of novel and unexpected recommendations) (Afsar et al., 2022). While a combination of collaborative filtering and content-based filtering is possible, this would only solve a portion of the issues that these systems face. Reinforcement learning (RL) seems like the perfect solution to this problem. RL approaches are able to handle dynamic environments and are meant to maximize long-term engagement with the system. There are many RL or deep reinforcement learning (DRL) approaches that can be used for recommender systems. RL-based methods include Monte Carlo, dynamic programming, and temporal difference methods. DRL methods typically include deep Q-network (DQN), double Q learning (Double DQN), dueling network,

and proximal policy optimization (PPO). Methods like the multi-armed bandit are not suitable for recommender systems because the environment is too dynamic (Afsar et al., 2022).

2.2 Q-Learning Approaches to Recommendation Systems

The most popular methods for implementing recommender systems are temporal difference methods such as Q-learning and Sarsa, according to Afsar et al. (2022). Dynamic programming methods are not well-suited to the problem of recommendation systems because they are computationally expensive and require perfect knowledge of the environment. Thus, temporal difference methods are often chosen due to their simplicity and positive results. Often, a decreasing learning rate of $\alpha = 1/1+\text{visits}(s, a)$ is used to guarantee algorithmic convergence. However, these methods' performance often suffers as the size of the state space increases, an issue known as the curse of dimensionality. To guarantee algorithmic convergence, researchers must ensure a small state space. It is also possible to leverage a neural network to estimate the value of each state-action pair, which is less bulky than assigning each state-action pair a value, but necessitates training and refinement. Q-learning has been successfully utilized by Chi et al., and Hu et al. to generate music recommendations, and our goal was to see if we could generate similar results for movie recommendations.

2.3 Deep Learning Approaches to Recommendation Systems

As mentioned in Section 2.2, traditional reinforcement learning approaches are often limited by an issue of scalability. When working with large state and action spaces, it can be difficult for an agent to learn how to make effective recommendations due to the complexity of the problem itself. In situations such as these, it can be beneficial to use deep reinforcement learning, which can find difficult and non-obvious relationships between the states (Afsar et al., 2022). Afsar et al. provide a summary of the three categories of methods within deep reinforcement learning: value-based, policy gradient, and actor-critic. Value-based methods such as Deep Q-learning (DQN) use a neural network to approximate the Q-value for a given state. Policy gradient methods such as REINFORCE use a parameterized policy and are capable of selecting actions without a value function. Actor-critic methods, such as deep deterministic policy gradient or proximal policy optimization have an actor which deterministically defines the value for a state while the critic provides an evaluation of the action taken. For our investigation, we chose to use a Double Q-Learning (Double DQN) approach and compare it with a DQN method. DQN has difficulties with bias in Q-value estimates due to an overestimation of values. As such, Double Q-Learning was developed. It has two estimators: one for action selection, and another for value estimation. This helps reduce the bias in Q-value estimates, which is why we have elected to use it. There have been successful uses of DQN and Double DQN for recommendation systems, such as Zhao et al., Fu et al., and Munemasa et al., but these approaches focus on recommendations in a broad sense, or applications such as online product recommendations. Movie recommendations can be similar, but not the same, as product

recommendations. Thus, we are interested in determining whether similarly positive results can be achieved with movie recommendations.

3 Technical Approach / Methodology / Theoretical Framework

3.1 Problem Formulation

We compared the performance of two different RL models: SlateQ and Double Q-Learning for personalized movie recommendations. Since we are not integrating our work with any external interface, we will be simulating our environment locally.

The problem of recommendations was formulated as:

- **State:** User ratings, movie attributes, user demographics
- **Action:** Based on given user rating and history, recommend a movie to a user and predict the order of ranking of preferred movies
- **Reward:** Dependent on the method
    - **Double DQN:** based on conventional reward practices and trials (+1 reward for any correct prediction from the recommendations given by the model, negative reward otherwise. Different values for negative reward were experimented with, finally we settled on -0.15 for 100k and -0.4 for 1m)
    - **SlateQ:** average of actual user ratings of movies in a slate

We chose our reward function to be based on what we think would maximize the learning for each method.

3.2 Dataset

For our dataset, we chose to use the MovieLens Dataset introduced by Harper et al (2015). The dataset was collected during the 90's and includes movie ratings rated on a scale of 1 to 5. The title, release date, IMDb URL, genre, and unique id for each movie is available as well. Each user in the dataset has rated at least 20 movies. There is further demographic information about each user, and includes age, gender, occupation, zip code, and the time the rating was made.

For our purposes, we used an 80/20 split for training and testing data. We chose to use the first 80% of a user's total ratings after sorting by the timestamp of each rating as the training set and reserve the remaining ratings for the test set. Splitting our data in this manner will allow us to assess whether the movies that the user ended up watching were consistent with what the recommendation system anticipated they would enjoy, based on their history.

Two versions of this dataset were utilized, one with 100,000 ratings and one with 1 million ratings. These versions will be referred to as MovieLens-100k and MovieLens-1M. MovieLens-100k has 100,000 collected ratings from 943 users on 1682 movies while MovieLens-1M has 1,000,000 ratings from 6,000 users and 4,000 movies. We tested our methods on both datasets in order to observe how the performance would scale. One thing to

note is that the training file for 100k Double DQN contains a little over 80% of the total data because we want the training data to contain all the movies in the dataset, otherwise the prediction accuracy is very low.

Sex of Users

Male Users	71.04%
Female Users	28.95%
Total Users	943

Ages of Users

Under 25	272
26-45	481
45+	190

Number of Films: 1682, across 216 genres
Most Popular Genres: Drama, Comedy, Comedy-Romance, Drama-Romance, Comedy-Drama
Most Often-Rated Films: Star Wars, Contact, Fargo, Return of the Jedi, Liar Liar

Figure 1: Statistics of MovieLens-100k

Sex of Users

Male Users	71.71%
Female Users	28.29%
Total Users	6040

Ages of Users

18-25	3199
26-45	1743
45+	870

Number of Films: 3706, across 306 genres
Most Popular Genres: Drama, Comedy, Horror, Comedy-Drama, Comedy-Romance
Most Often-Rated Films: American Beauty, Star Wars Episode IV, Star Wars Episode V, Star Wars Episode VI, Jurassic Park

Figure 2: Statistics of MovieLens-1M

3.3 SlateQ DQN Model

Our SlateQ agent is based on the work of Ie et al. which defines SlateQ as "a decomposition of value-based temporal-difference and Q-learning that renders RL tractable with slates." A slate refers to a list of recommendations, rather than a single-item recommendation. SlateQ states that the long-term value of a slate is equivalent to the long-term value of the component items. Slates are useful for minimizing the combinatorics of the action space,

especially in the context of recommender systems which consist of many users and items. Determining the long-term value of each possible slate is more difficult than determining the long-term value of each item composing the slate.

Each user is represented by a user profile vector: a vector with a length equal to the total number of movies rated. Each entry represents the user's rating for the movie with value 1 through 5. If the user has not seen the movie, the value is 0. At the start of each episode, a user is randomly selected and their profile is the initial observation. At each episode, the agent recommends a slate (consisting of a few movies) and the reward is calculated. The reward is defined as the mean rating the user has given to the movies in the slate. If the user has not seen any movie in the slate, no reward is added.

The agent is trained using standard Q-learning with function approximation via a neural network. We had originally planned to use a version of SlateQ that did not utilize deep learning in order to observe and compare its performance with double DQN. However, we determined that this was infeasible due to computational constraints. When we used a standard Q-table, we found no improvement over the course of over 100,000 episodes. Since we are working with a large state space, we realized we could not have a distinct entry in the Q-table for each user and still achieve good results, and so we used a simple neural network to approximate the Q-value for each state-action pair. User and movie embeddings were concatenated and passed through a fully connected layer to predict the expected reward for each user-item pair.

An epsilon greedy policy with a decaying epsilon (from 1.0 to 0.1 decreasing across all episodes) was used. The agent samples a random slate with probability epsilon, otherwise it selects movies with the highest scores from the Q-network's predictions. After taking an action, in this case looking at a slate, the Q-network is updated by minimizing the mean squared error (MSE) between the predicted Q-values and the actual reward as described previously. A grid search was performed to determine the best number of hidden layers for the Q-network, the learning rate, and the slate size. The best performing hyperparameters observed were 128 hidden layers, a slate size of 7, and an Adam optimizer with a learning rate of 0.00001.

All SlateQ experiments were run locally on a machine with a 1.2 GHz Quad-Core Intel Core i7 CPU, 16 GB of 3733 MHz LPDDR4X RAM, and integrated Intel Iris Plus Graphics (1536 MB). No external GPUs were utilized.

3.4 Deep Learning RL Model (Double Q-Learning)

For our second method, we implemented Double DQN (Hasselt et al., 2015). Double DQN is an extension of DQN. DQN is a type of reinforcement learning that uses a multi-layered neural network which outputs a vector of action values when given a state. The DQN was first introduced by Mnih et al. (2015), wherein the authors describe the two important ingredients of DQN: the target network, and experience replay. The target network is copied from the online network every n steps and remains the same for all other steps. The target function used by DQN is:

$$Y_t^{DQN} \equiv R_{t+1} + \gamma max_a Q(S_{t+1}, a; \theta_t^-) \quad (1)$$

The max function in equation (1) causes DQN to tend to select overestimated values in some cases which can cause overoptimistic estimates (Hasselt et al., 2015). To improve on this problem, Hassel et. al (2015) introduced double Q learning, or Double DQN. Double DQN uses an additional set of weights to update the target function, which is expressed in the following equation:

$$Y_t^{Double\ Q} \equiv R_{t+1} + \gamma Q(S_{t+1}, argmax_a Q(S_{t+1}, a; \theta_t); \theta_t^{'}) \quad (2)$$

The second set of weights $\theta_t^{'}$ in equation 2 can be updated by switching the roles of $\theta^{'}$ and θ. Double DQN results in more stable learning and is able to reduce the overoptimism in estimation outcomes. The implementation of the Double DQN is based on that of the architecture and deep neural network of the DQN algorithm and doesn't require additional networks or parameters (Hasselt et al., 2015).

4 Evaluation Metrics

Since the goal of our work was to evaluate the differences between SlateQ and Double DQN with regards to recommendations, we chose to evaluate performance through a variety of metrics and compare them against random baselines. The first was a top-k hit rate, which measured whether the movies recommended by the agent were part of the movies that the user watched later. Top-k hit rate calculates the percentage of users in which at least one correct item is recommended in the top k recommendations the agent produces. We used the reserved test set as our reference for the movies that each user ended up watching. This allows us to measure the success of our system, because if the agent is able to recommend the movies the user actually does watch later in time, it means that the agent's recommendations are successful. We compared the top-k hit rate for each method's agent to a system that selects recommendations at random. The random baseline for each model also slightly differed due to differences in data selection technique for the models (Most of the selection strategies are the same except for the 100k Double DQN as mentioned in section 3.2), but can still be compared by comparing their top-k hit rates. A k-value of 4 was utilized since there is a maximum of 4 movies that can be confirmed to be watched in the future by each user.

An additional second metric used is normalized discounted cumulative gain (nDCG). This measures whether the rankings of the movies is accurate, and is an indicator of how accurate the ratings are as opposed to the hit rate, which measures solely if the recommended movies were watched.

Our hypothesis is that the Double DQN will outperform the Slate DQN model, but perform worse in situations with limited resources or information. Since Double DQN depends

on two neural networks to generate recommendations, we predict that it will suffer in situations where data is less available. In these situations, we expect Slate DQN to perform better because it relies solely on optimizing one Q-function.

5 Results

5.1 SlateQ DQN Model

SlateQ performs about three times better than the baseline for both the 100k dataset and 1M dataset, but performs worse on the 1M dataset. The best SlateQ agent had a top-4 hit rate of 18% for the 100k dataset but 10% for the 1M dataset, suggesting that an increase in number of users and movies resulted in overall worse performance. This suggests that SlateQ may not be totally scalable, but still performs better than an agent that recommends movies randomly. Similarly the ability to accurately rank movies decreases, as the best NDCG@4 decreased from 12% to 6.7% from the 100k to 1M dataset.

	Top-4 Hit Rate	**NDCG@4**	**Training Time**
MovieLens-100k dataset			
SlateQ Agent (2,000 episodes)	0.07	0.038	32 seconds
SlateQ Agent (20,000 episodes)	0.15	0.10	6 minutes
SlateQ Agent (100,000 episodes)	0.17	0.12	21 minutes
SlateQ Agent (200,000 episodes)	0.18	0.12	64 minutes
SlateQ Agent (400,000 episodes)	0.17	0.12	89 minutes
Random Baseline	0.051	0.025	
MovieLens-1M dataset			
SlateQ Agent (100,000 episodes)	0.10	0.067	20 minutes
Random Baseline	0.021	0.0226	

Table 1: Performance of multiple configurations of SlateQ agent trained on MovieLens-100k and 1M. Improvement in performance plateaus 100,000 episodes with a training time of 21 minutes.

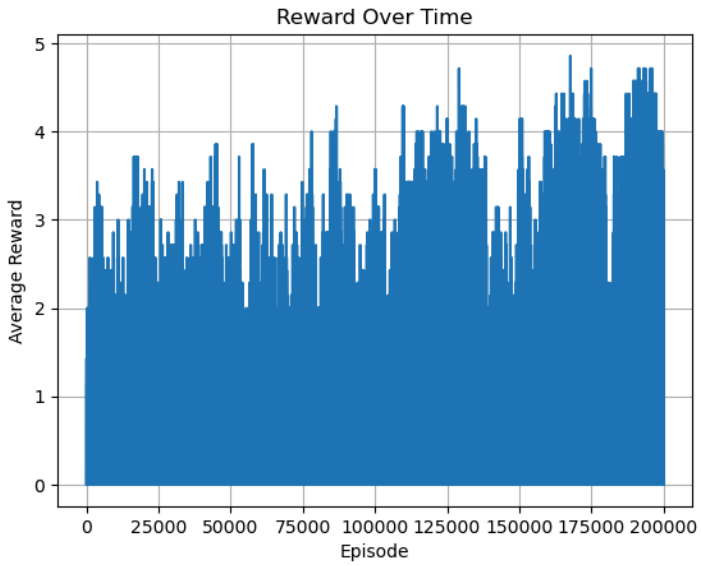

Figure 3: Reward over time of the training of a SlateQ agent on MovieLens-100k. The reward jumps to zero often due to many slates resulting in no reward because the user has not seen any movies in the slate.

5.2 Double DQN Model

The performance of the Double DQN model was not as high as we expected, possibly due to the fact that we didn't have enough time to finetune the parameters for the best performance. The data presented below are gathered from several different trials. The 100k experiment was run using a discount factor of 0.95, learning rate 0.0005, starting epsilon = 1, epsilon decay rate of 0.995, minimum epsilon 0.05, and a replay memory of size 200. The 1M experiment was run using the same parameters except for a replay memory size of 1000. In general, Double DQN brought about 1% of prediction accuracy increase. For the 100k dataset, the highest accuracy is 3.4%for top-4 and 3.6% for NDCG with 25 episodes and 50 iterations per episode. For the 1M dataset, the highest accuracy is 3.1% for top-4 with 50 episodes and 100 iterations per episode, the highest NDCG is 3.2%, coming from 100 episodes with 100 iterations per episode.

	Top-4 Hit Rate	**NDCG@4**	**Training Time**
MovieLens-100k dataset			
Double DQN (25 episodes, 50 iterations)	0.034	0.036	1.5 minutes
Double DQN (50 episodes, 50 iterations)	0.021	0.02	3 minutes
Double DQN (100 episodes, 100 iterations)	0.031	0.035	30 minutes
Random Baseline	0.0257	0.0274	
MovieLens-1M dataset			
Double DQN Agent (25 episodes, 50 iterations)	0.0205	0.023	3 minutes
Double DQN Agent (50 episodes, 100 iterations)	0.031	0.031	40 minutes
Double DQN Agent (100 episodes, 100 iterations)	0.03	0.032	90 minutes
Double DQN Agent (50 episodes, 200 iterations)	0.025	0.03	2 hours
Random Baseline	0.021	0.0226	

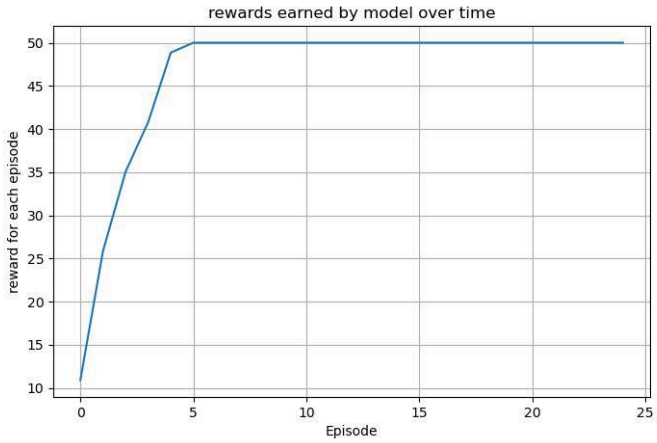

Figure 4: reward accumulated over time by Double DQN using the 100k dataset with 25 episodes, 25 iterations/episode

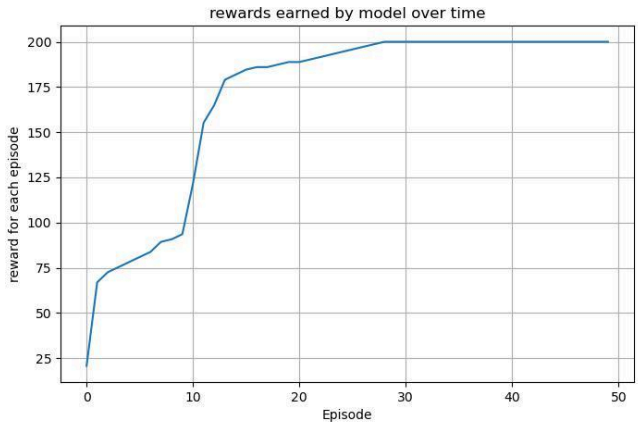

Figure 5: reward accumulated over time by Double DQN using the 1M dataset with 50 episodes, 200 iterations/episode

6 Discussion

As can be observed by the results in the tables in section 5, the accuracy of the two models differ quite significantly. Potential reasons leading to the poor performance of Double DQN could be due to lack of time to finetune the parameters as well as sparse reward learning to slower learning. It seems like it also takes more episodes for both agents to achieve higher accuracy in the 1M dataset. Another thing to note is that although Double DQN consistently performed worse than SlateQ, its performance is relatively stable across the two datasets compared to Slate Q, possibly due to the robustness of Double DQN. If given more time, we would have liked to perform further fine-tuning to maximize the prediction outcome, especially for Double DQN. The robustness of the models and their complexity were not something we anticipated. One particular aspect we wish to explore more is that if we could train the Double DQN agent on more episodes, could it achieve better results than Slate Q? The problem is that each run of the Double DQN takes a lot of time, so it's hard for us to run more episodes given the

computational resources we have. Overall, this experiment helped us showcase the performance of different RL models in the recommendation system, and shed light on a lot of the challenges that researchers might face when developing recommendation systems using RL.

7 Conclusion

In this paper, we presented our comparison of two different techniques to maximize user satisfaction with regards to media recommendations. Our work highlights the complexities and difficulties associated with developing such a system. We quickly found that as the quantity of users and media increased, the resources required to achieve satisfactory performance became exponentially greater. Given more time and resources, we would have liked to perform further tuning, and investigate other methods for media recommendations such as leveraging an autoencoder to transform input. We also would have liked to work with real-time user information. Since we were working with movie history that was not shaped by the recommendations we provided, it is difficult for us to determine the true efficacy of our work. However, we leave these investigations to future work. In conclusion, our work highlights the rich potential for applying reinforcement learning methodologies for media recommendation and the challenges in maximizing user satisfaction.

References

[1] Yu, Kathy, and Amy Wu. “Build a reinforcement learning recommendation application using Vertex AI.” *Google Cloud*, 18 August 2021, https://cloud.google.com/blog/topics/developers-practitioners/build-reinforcement-learning-recommendation-application-using-vertex-ai.

[2] Hasselt, Hado Van, et al. “Deep Reinforcement Learning with Double Q-learning.” *arXiv*, 8 December 2015, https://arxiv.org/pdf/1509.06461. Accessed 12 March 2025.

[3] F. Maxwell Harper and Joseph A. Konstan. 2015. The MovieLens Datasets:

[4] History and Context. ACM Transactions on Interactive Intelligent Systems (TiiS) 5, 4, Article 19 (December 2015), 19 pages. DOI=http://dx.doi.org/10.1145/2827872

[5] M. Mehdi Afsar, Trafford Crump, and Behrouz Far. Reinforcement learning based recommender systems: A survey, 2022.

[6] Binbin Hu, Chuan Shi, and Jian Liu. Playlist recommendation based on reinforcement learning. In ICIS’17, pages 172–182, 2017.

[7] Chi, Chung-Yi & Tsai, Richard Tzong-Han & Lai, Jeng-You & Hsu, Jane. (2010). A Reinforcement Learning Approach to Emotion-based Automatic Playlist Generation. 60 - 65. 10.1109/TAAI.2010.21.

[8] Xiangyu Zhao, Liang Zhang, Long Xia, Zhuoye Ding, Dawei Yin, and Jiliang Tang. Deep reinforcement learning for list-wise recommendations, 2019.

[9] Mingsheng Fu, Liwei Huang, Ananya Rao, Athirai A. Irissappane, Jie Zhang, Hong Qu. A Deep Reinforcement Learning Recommender System With Multiple Policies for Recommendations, 2023.

[10] Isshu Munemasa, Yuta Tomomatsu, Kunioki Hayashi, Tomohiro Takagi. Deep reinforcement learning for recommender systems, 2018.

[11] Eugene Ie, , Vihan Jain, Jing Wang, Sanmit Narvekar, Ritesh Agarwal, Rui Wu, Heng-Tze Cheng, Morgane Lustman, Vince Gatto, Paul Covington, Jim McFadden, Tushar Chandra, Craig Boutilier. "Reinforcement Learning for Slate-based Recommender Systems: A Tractable Decomposition and Practical Methodology." (2019).